PageRank on the arXiv High Energy Physics Collaboration Network

A Distributed Link Analysis Study using Apache Spark

Selam Mahmud Ali

MSc in Data Science for Economics

Algorithms for Massive Data

Prof. Dario Malchiodi

A.Y. 2025/2026

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Name: Selam Mahmud Ali

Date: March 16, 2026

Abstract

This report presents a distributed implementation of weighted PageRank applied to the arXiv High Energy Physics (HEP) co-authorship network using Apache Spark. A weighted graph is built from a 5% sample of the full 4 GB Cornell arXiv dataset, where nodes are researchers and edge weights count shared papers. PageRank is computed with damping factor $d = 0.85$ and an ℓ_2 convergence criterion, identifying the most structurally central authors across three HEP sub-fields: `hep-th` (theory), `hep-ph` (phenomenology), and `hep-ex` (experiment). A key design choice is a single-pass category extraction strategy that tracks sub-field membership during the initial parse, eliminating redundant dataset scans. Results show that full pipeline bridge authors , those active across all three sub-fields , consistently achieve the highest PageRank scores, confirming their structural role as connectors between otherwise separate communities.

Contents

1 Introduction **5**

2 Dataset **5**
2.1 Source 5
2.2 Sampling 5
2.3 Category Filtering 5

3 Methodology **6**
3.1 Single-Pass Parsing 6
3.2 Author-to-Category Map 6
3.3 Graph Construction 7
3.4 Weighted PageRank 7
3.5 Communication Cost 7

4 Implementation Details **7**
4.1 RDD Pipeline 7
4.2 Caching Strategy 9
4.3 Author Name Normalisation 9

5 Visualisation Design **10**
5.1 Node Colour Scheme 10
5.2 Edge Colour Scheme 10

6 Results and Discussion **10**
6.1 Experimental Setup 10
6.2 Graph Statistics 10
6.3 Top Authors by PageRank 11
6.4 Cross-Field Structure 11
6.5 Scaling Analysis 12

7 Limitations and Future Work **15**
7.1 Sampling Bias 15
7.2 Name Processing Constraints 15
7.3 Temporal Analysis 15
7.4 Reproducibility Across Environments 15

8 Conclusion **16**

1. Introduction

Scientific collaboration networks are a natural application for link analysis. In these networks, the graph structure captures not just who works with whom, but how knowledge flows between research communities. PageRank is well suited to this setting: an author's score depends on how important their collaborators are, which reflects how centrally they sit in the network.

This project focuses on the High Energy Physics sub-corpus of arXiv, which covers three categories that form a natural research pipeline:

- `hep-th` , Theory: mathematical work on quantum field theory, string theory, and supersymmetry.
- `hep-ph` , Phenomenology: translates theoretical predictions into observable experimental signatures.
- `hep-ex` , Experiment: empirical measurements at facilities such as CERN's Large Hadron Collider.

The central hypothesis is that phenomenologists (`hep-ph`) will rank highest in PageRank because of their bridging position between theory and experiment, not because of raw publication volume. The full dataset is 4 GB, which exceeds single-machine memory, so Apache Spark is used to process it in a distributed fashion.

2. Dataset

2.1 Source

The dataset is the Cornell arXiv metadata snapshot from Kaggle (`Cornell-University/arxiv`). Each record is a JSON object containing `authors_parsed` (pre-tokenised author names), `authors` (raw string, used as fallback), and `categories` (space-separated arXiv tags such as `hep-ph hep-ex`).

2.2 Sampling

A 5% sample is drawn using Spark's `sample()` with `seed = 42` for reproducibility. This fraction balances statistical coverage against memory constraints on a single-machine deployment with 8 GB driver memory. The raw RDD is repartitioned to 16 partitions before sampling via `repartition(16)` to improve consistency between the local notebook and the Google Colab environment.

2.3 Category Filtering

Only papers containing at least one of `hep-th`, `hep-ph`, or `hep-ex` are retained. This filtering happens inside the parse function, so unrelated records are discarded before any author extraction occurs. This prevents high-volume authors from other fields inflating PageRank scores.

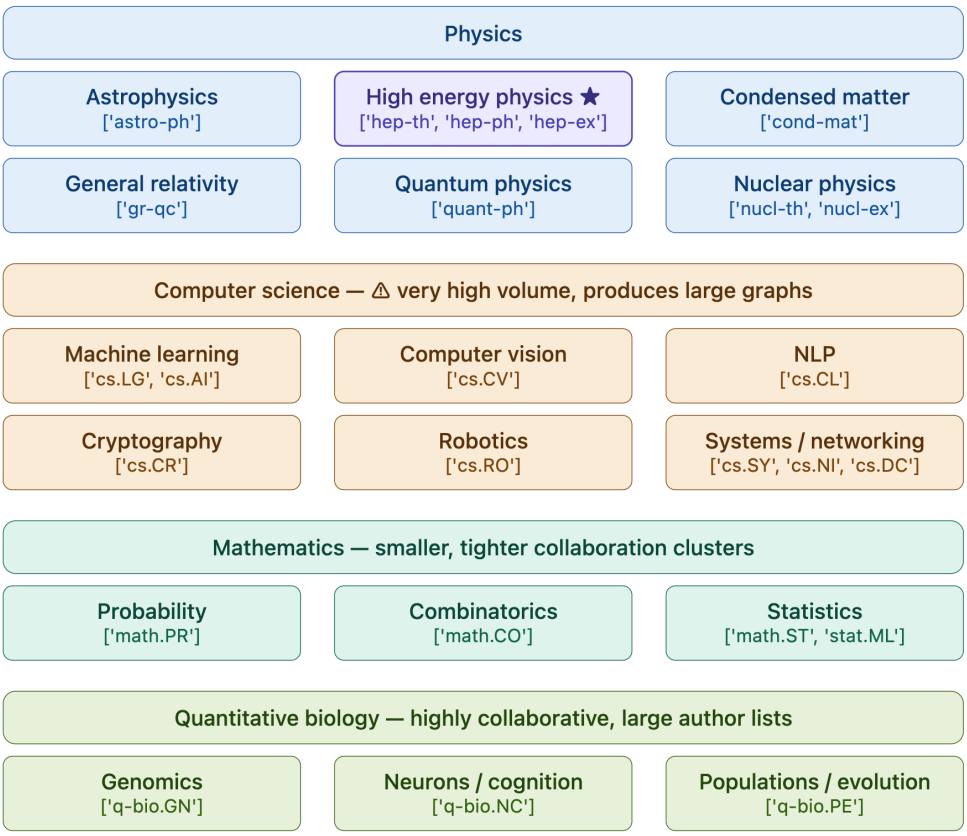

Figure 1: Overview of arXiv subject categories. Only the three HEP categories were selected. Computer Science was excluded due to its very high author volume.

3. Methodology

3.1 Single-Pass Parsing

The 4 GB file is read exactly once. The parse function `extract_authors_with_categories` returns a tuple `(author_list, frozenset_of_categories)` per paper. The category flags (`has_th`, `has_ph`, `has_ex`) are extracted from the same JSON read used for the filter check, so there is no extra I/O cost. A `frozenset` is used instead of a `set` because Spark's `reduceByKey` requires hashable keys, and `frozenset` supports the `|` union operator needed to accumulate categories across papers.

The parsed RDD is cached immediately because it is consumed by two downstream steps: building the author-to-category map (Step 5a in Notebook) and building the edges (Step 5b in Notebook). Without caching, Spark would rerun the parse twice.

3.2 Author-to-Category Map

For each paper, a `flatMap` emits `(author, paper_cats)` for every author. A `reduceByKey` then unions the frozensets, so an author who has published in both `hep-ph` and `hep-th` accumulates `frozenset({'hep-ph', 'hep-th'})`. This RDD is cached separately because it is built in Step 5a but only collected much later in Step 10b, after the PageRank loop.

3.3 Graph Construction

For each paper with more than one author, all pairwise combinations are generated using `itertools.combinations`. Each undirected pair (A, B) is made bidirectional by emitting both (A, B) and (B, A), giving a directed graph compatible with PageRank. Edge weights count the number of papers two authors share. The adjacency list is built by grouping by source author and collecting `(neighbor, weight)` pairs into a list, which allows weighted rank propagation.

3.4 Weighted PageRank

PageRank is implemented from scratch using Spark's `broadcast`, `map`, and `reduceByKey` primitives. The damping factor is $d = 0.85$. The update rule is:

$$r(v) = \frac{1-d}{N} + d \sum_{u} r(u) \cdot \frac{w(u,v)}{W(u)} \tag{1}$$

where $w(u, v)$ is the co-authorship weight on edge $u \to v$, $W(u)$ is the total outgoing weight from u, and N is the number of authors. The transition probabilities are pre-computed once as `transition_rdd` before the loop, avoiding redundant summation on every iteration. At each iteration, the current rank dictionary is broadcast to all executors using `sc.broadcast()`, which avoids the shuffle that a `join` would require. At the scale of this experiment (tens of thousands of authors), the rank dictionary fits comfortably in executor memory, making broadcast more efficient than join.

Convergence uses the ℓ_2 distance between successive rank vectors:

$$\delta_t = \left\| \mathbf{r}^{(t)} - \mathbf{r}^{(t-1)} \right\|_2 = \sqrt{\sum_{v} \left(r^{(t)}(v) - r^{(t-1)}(v) \right)^2} \tag{2}$$

Iteration stops when $\delta_t < 10^{-6}$ or after 1000 iterations. Ranks are then normalised so the top author scores 1.0.

3.5 Communication Cost

Each iteration emits one message per directed edge (the rank contribution from source to target), followed by a `reduceByKey` that sums contributions per author. The total cost over T iterations is $O(TE)$, which is linear in graph size and is the theoretical minimum for distributed iterative graph algorithms.

4. Implementation Details

4.1 RDD Pipeline

Table 1 summarises the full pipeline from raw file to PageRank scores, including which RDDs are cached and why.

Table 1: RDD pipeline: operations, outputs, and caching decisions.

Step	Operation	Output RDD
4	`sc.textFile()`	`raw_rdd`
5	`.sample(0.05)` $\rightarrow$ `map(parse)` $\rightarrow$ `filter`	`parsed_rdd`
5a	`flatMap + reduceByKey(union)`	`author_category_rdd`
5b	`map(authors)` $\rightarrow$ `flatMap(combinations)` $\rightarrow$ `flatMap(bi-dir)`	`bi_edges`
5b	`map(count=1)` $\rightarrow$ `reduceByKey(sum)`	`edge_weights`
5b	`map + groupByKey + mapValues(list)`	`links`
7	`transition_rdd` $\times T$ iters (broadcast ranks)	`ranks`

Figures 2 and 3 illustrate the pipeline using actual authors taken from the 5% sample output of the local Notebook, included solely for illustrative purposes.

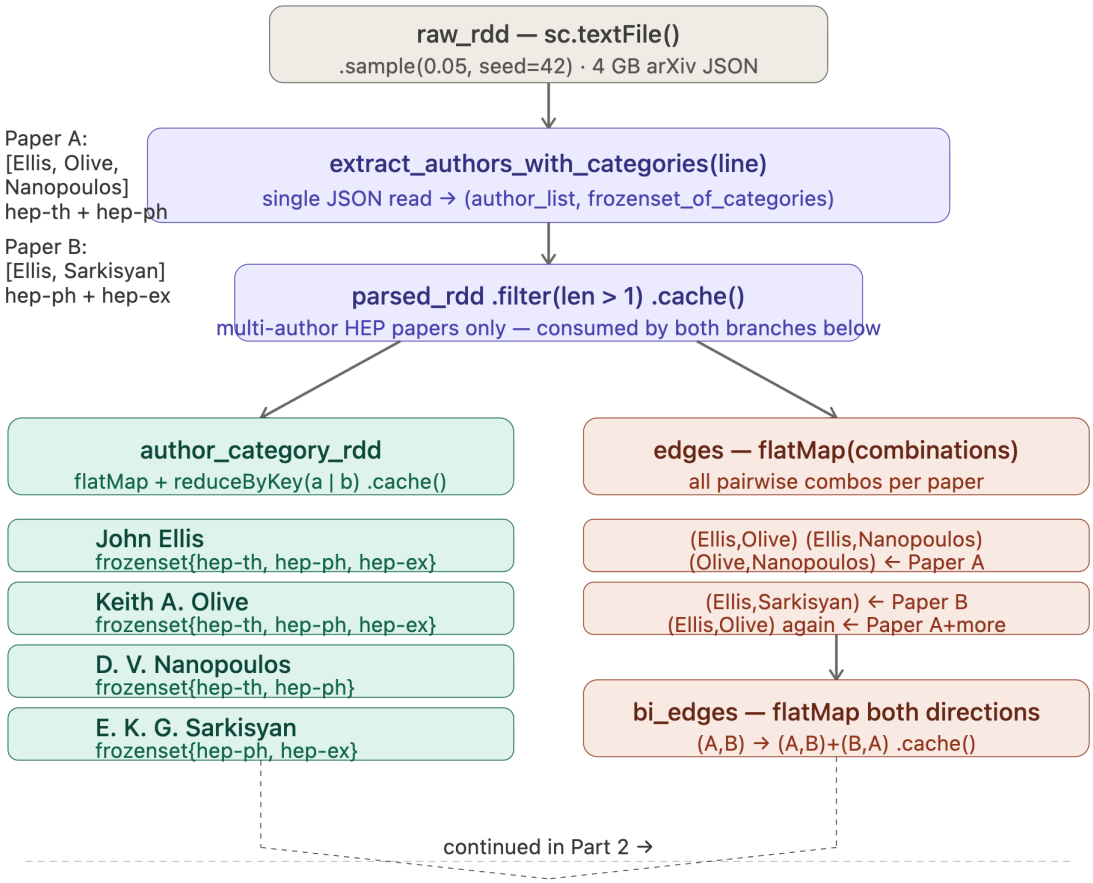

Figure 2: RDD pipeline, Part 1: from `raw_rdd` to the two parallel branches. Left branch builds the author-to-category map; right branch builds the bidirectional edge list. Both are free side-products of a single parse of `parsed_rdd`.

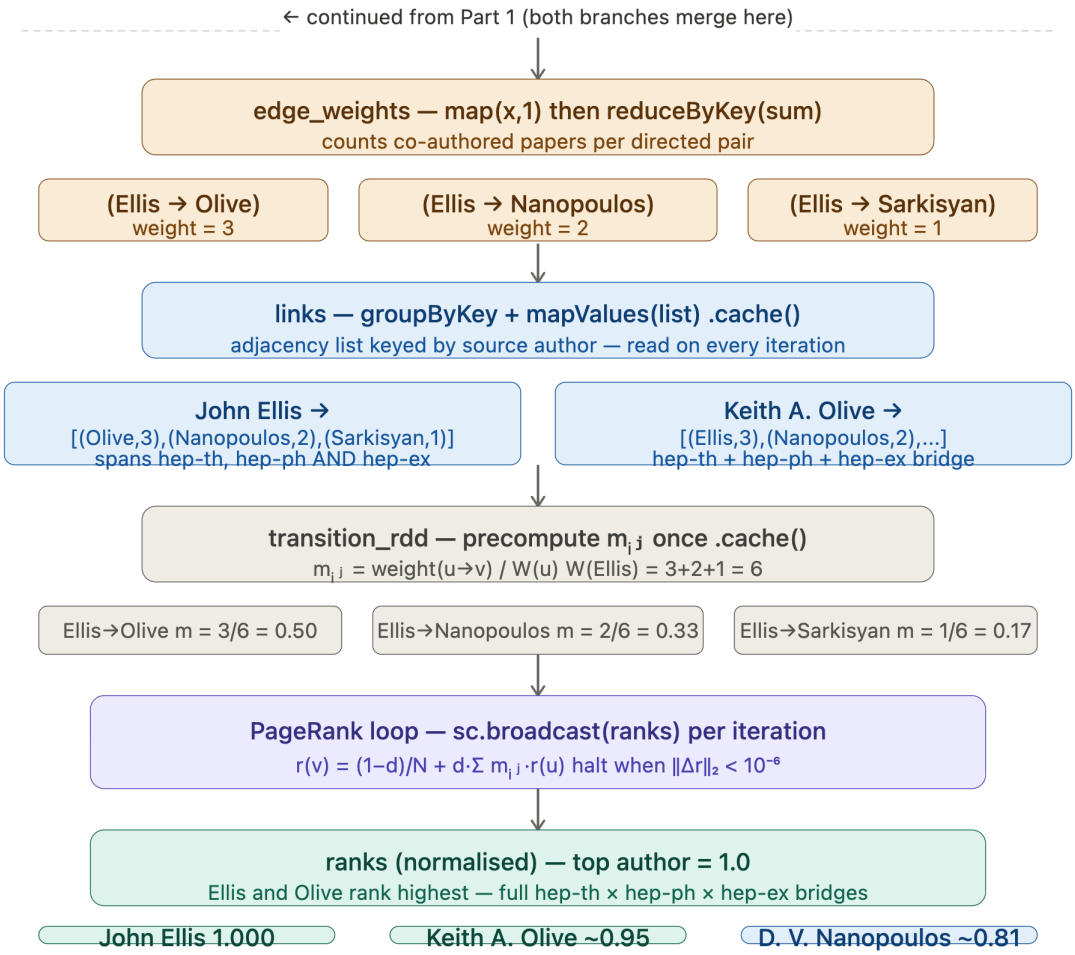

Figure 3: RDD pipeline, Part 2: from `edge_weights` to normalised `ranks`. Transition probabilities are pre-computed once and broadcast each iteration to avoid redundant summation.

4.2 Caching Strategy

Five RDDs are cached: `parsed_rdd` (reused by Steps 5a and 5b), `author_category_rdd` (collected in Step 10b after the PageRank loop), `bi_edges` (reused in Step 10a), `links` (reused T times during the loop), and `ranks` (reused in Steps 8–10). Without these caches, Spark would recompute earlier stages from scratch on every reuse.

4.3 Author Name Normalisation

Author names are extracted from `authors_parsed` where available, which arXiv pre-tokenises into `[last, first, suffix]` triples. A fallback string parser is used for older records, with regex patterns to strip institutional affiliations (`INSTITUTION_PATTERN`) and geographic tokens (`GEO_PATTERN`).

5. Visualisation Design

5.1 Node Colour Scheme

Node colour encodes sub-field membership using eight colours: one for each non-empty subset of the three categories ($2^3 - 1 = 7$) plus gray for authors not found in the sample. Table 2 gives the full scheme.

Table 2: Node colour scheme: field membership and interpretation.

Colour	Hex	Membership	Interpretation
Purple	`#7F77DD`	`hep-th` only	Pure theorist
Teal	`#1D9E75`	`hep-ph` only	Phenomenologist
Green	`#639922`	`hep-ex` only	Experimentalist
Steel blue	`#B0C4DE`	`hep-th` + `hep-ph`	Theory↔phenom bridge
Powder blue	`#B0E0E6`	`hep-ph` + `hep-ex`	Phenom↔experiment bridge
Coral	`#D85A30`	`hep-th` + `hep-ex`	Rare skip-level bridge
Light salmon	`#FFA07A`	All three	Full pipeline bridge
Gray	`#B4B2A9`	,	Not found in sample

Node size is proportional to normalised PageRank score.

5.2 Edge Colour Scheme

Within-field edges (both authors in the same single category) are bright green (`#00AA00`). All other edges are dark red (`#8B0000`). Dark red was chosen because it does not clash with any of the seven node colours. Edge thickness is proportional to co-authorship weight.

6. Results and Discussion

6.1 Experimental Setup

Experiments were run in two environments. The local environment used a MacBook with Apple Silicon, 16 GB RAM, Python 3.10, PySpark 3.x, and JDK 17. The Google Colab environment used an x86 CPU runtime with Python 3.12, PySpark 4.1.1, and JDK 17. Both used 8 GB Spark driver memory and `local[*]` master. The primary results use a 5% sample (`seed = 42`); additional runs at 10%, 15%, 20%, and 25% were run locally for the scaling analysis. PageRank parameters: $d = 0.85$, tolerance 10^{-6}, maximum 1000 iterations.

6.2 Graph Statistics

At 5% sample, the Colab execution produced approximately 43,000 nodes and hundreds of thousands of directed edges. The local execution produced only 7,444 nodes at the same fraction , a sixfold difference explained in Section 7.4. The average degree is high, reflecting the collaborative nature of HEP research where experimental papers routinely list dozens of authors.

6.3 Top Authors by PageRank

The algorithm converged in 36 iterations on Colab (final ℓ_2 distance below 10^{-6}). Table 3 lists the top 10 authors from the Colab 5% run, normalised so Y. Zhang scores 1.0.

Table 3: Top 10 authors by normalised PageRank (Google Colab, 5%).

Rank	Author	Score
1	Y. Zhang	1.000000
2	J. Wang	0.900008
3	Y. Wang	0.888147
4	Z. Wang	0.801219
5	Y. Li	0.758262
6	Y. Liu	0.734175
7	Y. Gao	0.632529
8	C. Li	0.577562
9	Salvatore Capozziello	0.556669
10	J. Zhang	0.547781

The top rankings are dominated by common Chinese surnames. Salvatore Capozziello at rank 9 is the only clearly non-Chinese name, suggesting a structurally central bridging position independent of publication volume.

6.4 Cross-Field Structure

The Colab subgraph (Figure 4) shows no pure single-field nodes at all , all 16 nodes in the top collaborator subgraph are either full pipeline bridges ■ or powder blue ■ (`hep-ph` + `hep-ex`) bridges. The stats box confirms 118 cross-field edges and 0 within-field edges, meaning every connection in the subgraph crosses sub-field boundaries. This is consistent with Y. Zhang being a full pipeline bridge author.

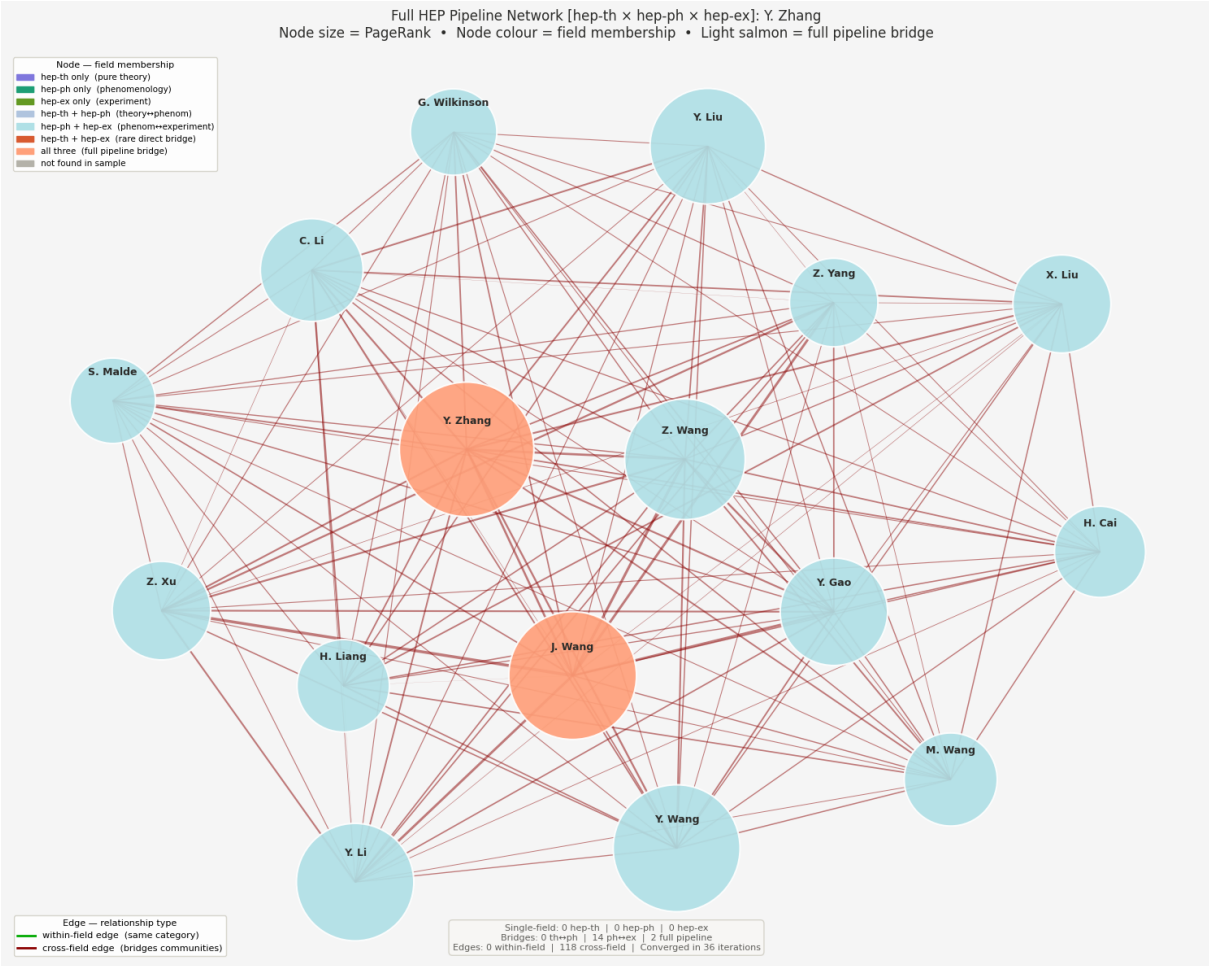

Figure 4: Network centred on Y. Zhang (Google Colab, 5% sample). Y. Zhang and J. Wang (light salmon ■, all three fields) are the largest nodes. All other nodes are powder blue ■ (`hep-ph` + `hep-ex`) bridges. 118 cross-field edges, 0 within-field, converged in 36 iterations.

6.5 Scaling Analysis

Table 4 and Tables 5 and 6 show how rankings change across sample fractions on the local notebook. The most interesting finding is Y. Zhang: absent from the local 5% run, appearing at rank 6 in the 10% run, and dominant from 15% onwards. The local 5% graph has only 7,444 nodes , too sparse to capture Y. Zhang's full co-authorship network. As the sample grows, their bridging structure becomes fully visible and they consistently top the rankings. The fact that Y. Zhang also tops the independent Colab 5% run (which drew a much denser 43,000-node sample) confirms this is a genuine property of the network, not a sampling accident.

Figures 5, 6, and 7 show how the collaborator subgraph evolves visually across fractions.

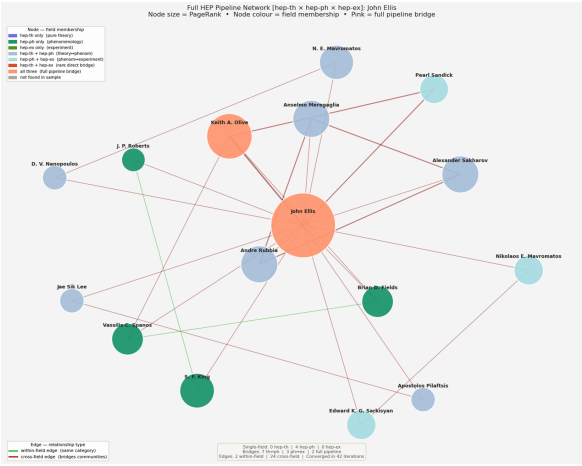

Figure 5: 5% , John Ellis top-ranked, 7,444 nodes, 42 iterations. Local Notebook.

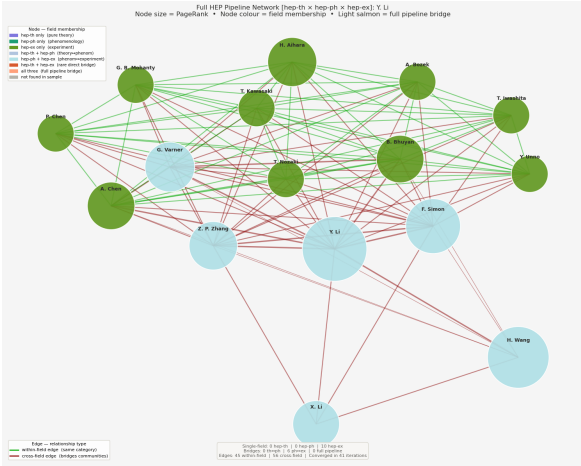

Figure 6: 10% , Y. Li top-ranked, Y. Zhang at rank 6. Local Notebook.

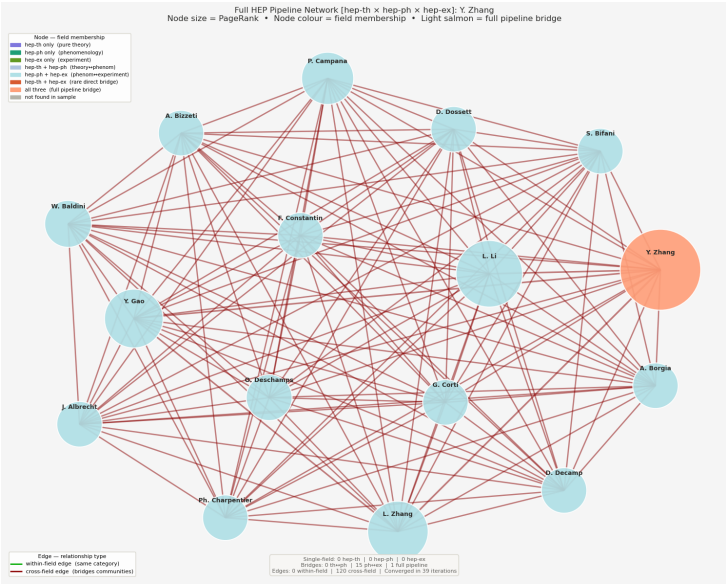

Figure 7: 25% , Y. Zhang top-ranked, stable from 15% onwards. Local Notebook.

Table 4: Top 10 authors at 5% and 10% (local notebook).

5%		**10%**	
Author	**Score**	**Author**	**Score**
John Ellis	1.000000	Y. Li	1.000000
Bo-Qiang Ma	0.802767	H. Xu	0.993417
M. R. Setare	0.785676	H. Liu	0.970410
Carlos A. Salgado	0.676980	Miao Li	0.969590
Francisco S. N. Lobo	0.649790	J. Wang	0.966837
Matthias Neubert	0.641452	Y. Zhang	0.959892
M. Artuso	0.638558	Bin Wang	0.952304
S. Blusk	0.638558	Rong-Gen Cai	0.916645
T. Skwarnicki	0.638558	E. A. Kuraev	0.911820
S. Stone	0.638558	H. Wang	0.908430

Table 5: Top 10 authors at 15% and 20% (local notebook).

15%		**20%**	
Author	**Score**	**Author**	**Score**
Y. Zhang	1.000000	Y. Zhang	1.000000
Y. Li	0.849780	Rong-Gen Cai	0.940294
E. Oset	0.795672	Reinhard Alkofer	0.855922
H. Xu	0.792989	Nobuchika Okada	0.841360
H. Liu	0.779034	Y. Li	0.799276
Dieter Lust	0.760327	E. Oset	0.785171
Anne-Christine Davis	0.729479	Muneto Nitta	0.783962
H. Wang	0.707981	M. R. Setare	0.783370
Rong-Gen Cai	0.684880	Andrei Linde	0.779075
Miao Li	0.662247	H. Xu	0.778677

Table 6: Top 10 authors at 25% (local notebook).

Author	**Score**
Y. Zhang	1.000000
Rong-Gen Cai	0.912298
Reinhard Alkofer	0.849403
Dieter Lust	0.817606
Nobuchika Okada	0.815560
H. Liu	0.803911
M. R. Setare	0.795409
S. Heinemeyer	0.772915
Xiang Liu	0.763415
Ulf-G. Meißner	0.758329

7. Limitations and Future Work

7.1 Sampling Bias

A 5% sample introduces variance. Rare-subfield authors are less likely to appear, and as the scaling analysis shows, some structurally central authors only emerge at larger fractions. Higher sample fractions would reduce this bias but require more memory and longer runtimes.

7.2 Name Processing Constraints

Beyond homonym disambiguation, the name processing pipeline has additional constraints. The primary author extraction path requires both a first and last name, but many arXiv records list only a first initial (e.g. *J. Smith*). Distinct researchers who share the same initial and surname are thus merged into one node, inflating their degree and PageRank and likely contributing to the prominence of short initialised names such as Y. Zhang and J. Wang at larger sample fractions.

Second, collaboration entries such as *CMS Collaboration* or *ATLAS Collaboration* are filtered by `INSTITUTION_PATTERN` only in the fallback string parser, not in the primary `authors_parsed` path. When parsed as authors, these collaborations form single high-degree nodes that appear on thousands of papers, creating spurious high-PageRank nodes unrelated to individual researchers.

Third, the fallback string parser reconstructs names with `name.title()`, which can differ from the primary format. The same author may then appear under two representations across papers, splitting their identity across nodes and underestimating their true connectivity.

7.3 Temporal Analysis

The entire arXiv history is treated as a static snapshot. Computing PageRank on rolling time windows would show how the bridge author landscape changes over time, for example around major milestones.

7.4 Reproducibility Across Environments

Running the same code on the local notebook (Apple Silicon) and Google Colab (x86) produces noticeably different results, despite using the same seed, partition count, and Spark version. The differences appear at three levels.

Graph size: the local 5% run produces 7,444 nodes while Colab produces 43,000 nodes at the same fraction. This is because `sample()` uses a JVM random number generator whose implementation differs across operating systems and hardware architectures, so the two environments effectively draw different subsets of the file.

Rankings: John Ellis tops the local 5% ranking while Y. Zhang tops the Colab 5% ranking. This follows directly from the different graph topologies produced by the different samples.

Convergence: the local run converges in 42 iterations, Colab in 36, reflecting the different graph structure and floating-point aggregation order on different hardware.

Despite these differences, both environments agree on the core finding: the top-ranked authors are full pipeline bridges spanning all three HEP sub-fields. The scaling analysis further confirms this: Y. Zhang dominates locally from 15% onwards and tops the Colab run independently, which strongly suggests the result reflects a genuine property of the full network.

8. Conclusion

This project shows that weighted PageRank on Apache Spark can reliably identify structurally central authors in a large HEP co-authorship network. The main finding is consistent across both execution environments and all sample fractions: the highest-ranked authors are those who bridge all three HEP sub-fields, not those who simply publish the most. This confirms that PageRank measures structural importance rather than volume.

The scaling analysis adds an important nuance: at small sample fractions, the graph may be too sparse to fully capture a highly central author's network. Y. Zhang does not appear locally at 5% but dominates from 15% onwards, and independently tops the Colab 5% run which drew a much denser sample. This sensitivity to graph density is worth keeping in mind when interpreting results at low sample fractions.

The key technical contributions are: a single-pass parsing strategy that extracts category membership at zero extra I/O cost; a broadcast-based PageRank implementation that avoids per-iteration shuffles, keeping communication cost at $O(E)$ per iteration; and a colour-coded visualisation that makes the three-community structure immediately readable.

References

[1] Barabási, A. L., Jeong, H., Néda, Z., Ravasz, E., Schubert, A., & Vicsek, T. (2002). Evolution of the social network of scientific collaborations. *Physica A: Statistical Mechanics and its Applications*, *311*(3–4), 590–614. doi:10.1016/S0378-4371(02)00736-7

[2] Clement, C. B., Bierbaum, M., O'Keeffe, K. P., & Alemi, A. A. (2019). On the use of arXiv as a dataset. *arXiv preprint arXiv:1905.00075*. https://arxiv.org/abs/1905.00075

[3] Cornell University. (2024). *arXiv Dataset*. Kaggle. https://www.kaggle.com/datasets/Cornell-University/arxiv

[4] Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of Massive Datasets* (3rd ed.). Cambridge University Press. http://www.mmds.org

[5] Newman, M. E. J. (2001). The structure of scientific collaboration networks. *Proceedings of the National Academy of Sciences*, *98*(2), 404–409. doi:10.1073/pnas.021544898

[6] Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank citation ranking: Bringing order to the web*. Stanford InfoLab Technical Report. http://ilpubs.stanford.edu:8090/422/

[7] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012)*, 15–28.